大模型训练前置知识

> 零基础友好的大模型分布式训练入门指南。读完你就能理解训练系统的核心挑战、并行策略、主流框架，以及如何让大模型在上千张 GPU 上稳定跑起来。

> **撰写时间**：2026-04-30

> 🎯 **读完这篇你能**：解释大模型训练的三大瓶颈（显存、算力、通信），区分数据并行/模型并行/流水线并行/序列并行的适用场景，并说出 ZeRO、混合精度训练、梯度检查点、FlashAttention 等核心优化各自解决什么问题。

一、训练 vs 推理：两种完全不同的工程挑战

很多人以为训练和推理是一回事——都是把模型跑一遍。实际上这两个阶段的目标、瓶颈、优化方向完全不同。

训练的本质是学习参数。模型读取海量数据，通过前向传播算预测、通过反向传播算梯度、更新参数。这个过程不在乎延迟——跑一天、一周、一个月都行，目标是 loss 降到最低。训练的瓶颈是显存和通信：你可能要同时调度上千张 GPU，每张 GPU 上的显存都不够存完整模型，GPU 之间还得频繁同步。

推理的本质是用已经学好的参数。参数固定，只需一次前向传播，没有反向传播。推理的瓶颈是延迟和吞吐：用户按了回车键就得尽快看到第一个字，还得同时服务几百几千人。

打个比方：**训练是盖房子，推理是住进去**。

盖房子需要大量材料、工人、设备协同工作，互相之间要协调进度，工期按月计。住进去之后关注的是住得舒不舒服、冷不冷、水电好不好用——跟盖房时的挑战完全不同。

维度	训练	推理
做什么	反向传播调参数	前向传播算结果
目标	loss 降到最低	延迟最低、吞吐最高
算力需求	极高（成千上万 GPU x 数周）	中等（单卡到几十卡）
显存需求	极高（参数+梯度+优化器状态+激活值）	中（参数+KV cache）
主要瓶颈	显存不够、通信太慢	延迟太高、显存带宽不足
精度要求	FP32/BF16/FP8 混合	可降级到 INT8/INT4

理解这个区别后，下面所有内容才有了意义——训练系统的一切努力，都是为了在"几十亿参数、上千张 GPU、跑几周到几个月"的约束下，让每一步计算又快又稳。

二、训练的三个核心资源

大规模训练消耗三种资源：算力、显存、通信。一个训练系统的好坏，就看它能在这三个瓶颈上做到什么程度。

算力（FLOPs）

GPU 的本质就是一台做矩阵乘法的机器。H100 的 FP16 算力约 1979 TFLOPS（NVIDIA, 2022），B200 约 4500 TFLOPS（NVIDIA, 2024）。你需要的总计算量 = 模型参数 x 训练 token 数 x 6（前向+反向各三倍），这是理论下限。训练总时间 = 总计算量 /（总算力 x GPU 利用率）。利用率的常见值是 30%–50%（社区估算）——通信瓶颈、显存搬运、调度开销都会让 GPU 空转。

算力通常是够的，真正缺的是显存和带宽。

显存（Memory）

这是训练的第一瓶颈。训练时每张 GPU 上要同时放这些东西：

- **模型参数**：模型本身。LLaMA 70B 在 FP16 下 140GB，一张 H100 80GB 放不下
- **优化器状态**：Adam 的 momentum + variance。参数量 x 2 倍（FP32），70B 就是额外的 560GB
- **梯度**：跟参数一样大
- **激活值**：前向传播中每层的中间结果，反向传播时重新计算要用。这是最大的"隐形"开销，跟 batch size 和序列长度成正比

通信（Communication）

当模型跨多张 GPU 时，GPU 之间必须同步数据。同步量取决于并行策略：数据并行每次迭代要同步整个梯度（几 GB），张量并行每层要同步几次（几百 MB），专家并行要跨卡路由 token。

通信带宽决定了你能用多少张 GPU 而不浪费。NVLink 单方向 900 GB/s（NVIDIA, 2022），普通以太网只有 25–100 GB/s（社区估算）。所以同节点内多用 NVLink，跨节点少通信。

如果把训练比作盖房子：算力是工人的效率，显存是材料仓库的大小，通信是工人之间的传话速度。仓库不够大（模型放不下），工人再多也白搭；传话太慢（通信瓶颈），大部分时间花在等人回话上。

三、数据并行（Data Parallelism）：最直观的并行

数据并行是最早、最简单的并行策略。一句话描述：**每张 GPU 都存一份完整的模型，各自处理不同的数据批次，最后把梯度同步一下**。

工作流程

```
数据分片
  ├── Batch 0 → GPU 0（完整模型）→ 前向 → 反向 → 梯度
  ├── Batch 1 → GPU 1（完整模型）→ 前向 → 反向 → 梯度
  ├── Batch 2 → GPU 2（完整模型）→ 前向 → 反向 → 梯度
  └── Batch 3 → GPU 3（完整模型）→ 前向 → 反向 → 梯度
                          ↓
                    All-Reduce 同步梯度
                          ↓
                  每张 GPU 用平均后的梯度更新自己的参数
```

All-Reduce 是所有 GPU 把自己的梯度拿出来求和取平均，再把结果分发给每个人。4 张卡时每张卡贡献 1/4。

优点

- **实现简单**：PyTorch 的 `DistributedDataParallel` 一行代码就能跑
- **计算完全独立**：前向和反向完全在本地完成，只有梯度同步需要通信
- **对通信带宽要求不高**：每步只同步一次梯度

缺点

- **显存浪费**：每张 GPU 都保存完整模型 + 优化器状态 + 梯度。4 张卡的集群，模型占的内存翻 4 倍——每张卡都在重复存相同的东西
- **batch size 过大的问题**：4 张卡 = batch size 变大 4 倍。太大会影响收敛质量，需要更多 warmup steps

数据并行对小模型（<1B）完全够用。但模型一大，显存冗余的问题就没法忍了。

打个比方：数据并行就像 4 个人各抄一本书，每人都拿一整套书和文具，但只抄自己分到的段落。最后对一下各自写的内容——资源浪费在每个人手里都有一套完整的书。

四、ZeRO：把"每人一份"变成"每人一部分"

ZeRO（Zero Redundancy Optimizer）是微软 DeepSpeed 的核心贡献（Rajbhandari et al., 2020）。它解决数据并行最大的浪费：**既然每张卡只算一部分数据，为什么每张卡要存完整的模型和优化器状态？**

ZeRO 的核心思想很简单：**大家别各自存完整的了，每人只存一部分，需要用时从别人那里取。**

三个递进阶段

阶段	分什么	省多少	通信量
ZeRO-1	分优化器状态	4x（Rajbhandari et al., 2020）	几乎没有额外通信
ZeRO-2	分优化器状态 + 梯度	8x（Rajbhandari et al., 2020）	稍有增加
ZeRO-3	分优化器状态 + 梯度 + 参数	16x（Rajbhandari et al., 2020）	显著增加

ZeRO-1：Adam 优化器存 momentum 和 variance，这两项的大小是模型参数的 2 倍（FP32）。原先每张卡各存一份。现在 N 张卡各存 1/N。省约 4x 显存（Rajbhandari et al., 2020），几乎不增加通信成本。

ZeRO-2：梯度也分掉。原先每张卡算完梯度后做 all-reduce，每张卡都拿到完整的梯度更新。现在只做 reduce-scatter（分发出平均值），每张卡只拿自己负责的那部分梯度。省约 8x 显存（Rajbhandari et al., 2020）。

ZeRO-3：模型参数也分掉。前向传播时，需要哪层参数就从对应的 GPU 取过来。算完就丢回去。省约 16x 显存（Rajbhandari et al., 2020），但每层都需要一次 gather 操作，通信量显著增加。

关键理解

ZeRO 不是模型并行。计算仍然在每张 GPU 上独立进行——每张 GPU 处理自己的 batch，完成完整的前向和反向。只是存的东西不再完整了。ZeRO-3 虽然会跨卡取参数，但计算过程跟单卡完全一样。

ZeRO-3 能让一张 H100（80GB）训练 175B 的 GPT-3，这在纯数据并行下是不可能的（Rajbhandari et al., 2020）。

打个比方：数据并行是每人一本完整字典，ZeRO 是把字典拆成几本分册，每人只拿一册。查到需要的那一页（参数）时，去拿着那册的人那里借来看一眼。

五、张量并行（Tensor Parallelism / TP）：切分一层

数据并行切数据，ZeRO 切存储，张量并行切计算本身。TP 把一个层的计算拆到多张 GPU 上，每张 GPU 只做 1/N 的计算。

怎么切

以 Feed-Forward Network（FFN）为例。FFN 的计算是 Y = Activation(X * W)，其中 W 是一个大矩阵。

- **列切分**：把 W 竖着切成 N 份，每张 GPU 拿一份，各自做 X * W_i。结果拼起来
- **行切分**：把 W 横着切成 N 份，每张 GPU 拿一份。输入 X 也切。用 all-reduce 合并结果

```
完整 FFN 计算：
        X × W → Activation → 输出

TP（2 张 GPU）：
GPU 0:  X × W_左 → 部分结果
GPU 1:  X × W_右 → 部分结果
                ↓
          All-Reduce 合并结果
                ↓
          Activation → 完整输出
```

对 Attention 也是一样：把多头注意力中的"头"切分到不同 GPU 上，每张 GPU 只算几个 head，最后合并。

通信要求

TP 的问题是**通信极其密集**。每一层、每次矩阵乘法之后都要 all-reduce。如果模型有 80 层，一层里有两个矩阵乘法（QKV + 输出投影 + FFN 两个），一次前向就要做几百次 all-reduce。

这意味着 TP 只能在 NVLink 或 NVSwitch 连接的多张 GPU 上使用（900 GB/s），不能用普通以太网跨节点。

与 ZeRO 的区别

ZeRO 每张 GPU 存一部分参数，但算的是完整的矩阵乘法（参数取过来再算）。TP 每张 GPU 不仅存一部分参数，也只算一部分，最后靠通信把结果拼起来。

ZeRO 省存储不省算力，TP 省显存也省每张卡的计算量。

实际中，TP 通常配合 NVLink 在单节点内用（8 卡节点内部 TP=8），跨节点之间用 ZeRO 或 PP。

六、流水线并行（Pipeline Parallelism / PP）：分层放置

张量并行是把一层切到多张卡上。流水线并行是把不同层放到不同卡上。

基本原理

```
模型共 40 层（Layer 0-39）

GPU 0: Layer 0-9
GPU 1: Layer 10-19
GPU 2: Layer 20-29
GPU 3: Layer 30-39

前向传播：Batch → GPU0 → GPU1 → GPU2 → GPU3 → 输出
反向传播：输出 ← GPU3 ← GPU2 ← GPU1 ← GPU0 ← Loss
```

数据从 GPU 0 一层层传到 GPU 3，每一张卡只计算自己那 10 层。反过来反向传播。

气泡问题（Pipeline Bubble）

PP 最大的问题是**气泡**（Huang et al., 2019）。假设有 4 个 GPU（4 个 stage），micro-batch 有 8 个：

- GPU 0 先处理前 8 个 micro-batch 的前向（耗时 8 个单位）
- GPU 0 把结果传给 GPU 1，GPU 1 开始算（耗时 8 个单位）
- GPU 2 要等 GPU 1 算完才能开始（又等 8 个单位）
- GPU 3 要等 GPU 2

等 GPU 3 开始推理时，GPU 0 已经空闲了 24 个时间单位。这些空闲时间就是"气泡"。

1F1B（One Forward One Backward） 调度是常用的缓解方案（Huang et al., 2019）：不用等所有 micro-batch 前向做完再统一反向，而是在做的过程中穿插反向，让上下游尽快有空位接新的 micro-batch。

PP 的优缺点

优势：

- 通信量小（每张卡之间只传激活值，几 MB 级别）
- 普通以太网够用

劣势：

- 气泡问题无法完全消除
- 需要在层边界切分，每张卡分的层数要均衡

TP 适合单节点内（高带宽），PP 适合跨节点（低带宽）。TP 在时间上拆分计算，PP 在深度上拆分模型。

七、3D 并行（三维并行）：TP + PP + DP 的组合拳

单个并行策略都有各自短板。DP 浪费显存，ZeRO 增加通信，TP 带宽要求高，PP 有气泡。对于 100B+ 的模型，业界公认的解法是**三种并行一起用**——这就是 3D 并行，由 Megatron-LM 推广开来（Shoeybi et al., 2020）。

分层结构

```
          数据并行组（跨节点，以太网）
┌──────────────────────────────────┐
│        节点 0                    │        节点 1
│   ┌─ TP 组 ─┐                    │   ┌─ TP 组 ─┐
│   │ GPU 0-3 │  (NVLink)          │   │ GPU 4-7 │
│   └─────────┘                    │   └─────────┘
│        ↑ PP                      │        ↑ PP
│   ┌─ TP 组 ─┐                    │   ┌─ TP 组 ─┐
│   │ GPU 4-7 │  (NVLink)          │   │ GPU 0-3 │
│   └─────────┘                    │   └─────────┘
└──────────────────────────────────┘
```

在这个架构里：

- **TP 在组内（GPU 0–3 / GPU 4–7）**：4 张卡通过 NVLink 组成 TP 组，共享一层内计算。通信最密集，带宽最高
- **PP 在组间（同一节点内不同 TP 组）**：模型的不同层放在不同 TP 组上。通信量小，普通链路即可
- **DP 在节点间（节点 0 ↔ 节点 1）**：不同节点处理不同 batch，只同步梯度。跨节点以太网通信

典型配置

模型规模	节点数	每节点 GPU	TP	PP	DP
175B	16	8	8	4	4
530B	64	8	8	8	8
1T	128	8	8	16	8

典型 3D 并行配置（Shoeybi et al., 2020）

TP=8+PP=8+DP=8 共 512 张 GPU：TP 负责压单卡显存，PP 解决跨节点长距离通信问题，DP 叠加吞吐量。

工程复杂度

3D 并行不是三个策略的简单叠加。它需要精确编排：什么时候做 TP 的 all-reduce，什么时候做 PP 的微批次调度，什么时候做 DP 的梯度同步。每一步的通信和计算要尽量重叠，不能让 GPU 闲着等数据。

这也是为什么 Megatron-LM 比 FSDP 难上手但性能更极致——它把三层并行的手动调度做到了最优。

八、FSDP：PyTorch 的统一方案

FSDP（Fully Sharded Data Parallel）是 PyTorch 从 ZeRO-3 获取灵感推出的原生方案。它的目标很简单：**让大家不用深入理解并行细节就能训练大模型。**

核心思想

FSDP 做的事情跟 ZeRO-3 一模一样：把所有东西（参数、梯度、优化器状态）都分片到所有 GPU 上，需要时再集中起来。

但 FSDP 的 API 设计远胜 DeepSpeed：

```
from torch.distributed.fsdp import FullyShardedDataParallel as FSDP

model = FSDP(model)
# 就这样，不需要手动配置任何并行策略
# FSDP 自动决定何时 gather 参数、何时 reduce-scatter 梯度
```

在训练循环中，FSDP 自动做了这些事情：

1. 前向传播时：gather 需要的参数分片 → 计算 → 释放其他 GPU 的参数
2. 反向传播时：重新 gather 参数算梯度 → reduce-scatter 梯度到对应 GPU
3. 优化器更新：每张 GPU 只更新自己保存的参数分片

FSDP vs Megatron

维度	FSDP	Megatron-LM
使用门槛	低，几行代码	高，需要理解并行策略
并行能力	FSDP（ZeRO-3）	TP + PP + DP + EP
通信效率	自动管理	手动优化到极致
性能	比 Megatron 低 5-15%（社区估算）	当前最高
适用团队	研究组、小团队	大厂 infra 团队
生态	PyTorch 原生，生态最广	需要定制

什么时候选 FSDP

- 模型在 1B-70B 之间
- 团队只有几个工程师
- 想快速实验和迭代
- 不需要 TP（单节点内用 FSDP 就够了）

FSDP 已经是中小团队训练大模型的默认选择。对于绝大多数场景，它的性能差可以接受，但开发效率的提升巨大。

九、训练稳定性：Loss 不降怎么办

训练大模型最常见的问题不是性能不够，而是训练崩了。Loss 突然变 NaN、模型不收敛、训到一半梯度爆炸——这些问题在大规模训练里几乎每天都会遇到。

常见问题排查清单

Loss 发散 / 变成 NaN

- 梯度爆炸：加 gradient clipping，设 max_norm=1.0
- 学习率过高：调低 LR 或延长 warmup
- 数据中混入了 Inf/NaN：检查数据 pipeline
- Adam 中 epsilon 太小：从 1e-8 提到 1e-6

Loss 不降或降得太慢

- 学习率太低：试试余弦退火加 warmup
- batch size 太大：大 batch 需要同步更多的 warmup steps
- 数据问题：标签对不对？数据分布跟预训练是否一致？
- 模型初始化：检查权重初始化策略

训练早期 loss 飙升

- warmup steps 太短：前几百步梯度方向混乱，需要逐步提高 LR
- 模型架构问题：检查 normalization 的位置和类型

显存不足（OOM）

- batch size 太大：减小 micro-batch size
- 序列太长：检查梯度检查点（activation checkpointing）是否开启
- 激活值太大：开启 activation recomputation（用 30% 计算换显存）

梯度检查点（Activation Checkpointing）

这是一种"算力换显存"的技术。前向传播时**不保存**中间激活值（通常占显存大头），反向传播时**重新计算**它们。代价是额外 30-40% 的计算量，好处是显存节省 3-5 倍（社区估算）。

几乎所有大模型训练都默认开启。

混合精度训练（Mixed Precision）

深度学习训练中一个有趣的矛盾：模型精度不需要很高（BF16/FP16 就够），但优化器状态需要很高（FP32 以应对小梯度更新）。

混合精度训练的做法是（Micikevicius et al., 2018）：

- 前向和反向用 FP16 或 BF16（省显存、快）
- 优化器状态用 FP32（保精度）
- 梯度从 FP16 转成 FP32 后更新参数

BF16 比 FP16 更受欢迎，因为它对精度要求不敏感——训练早期 BF16 就能稳定收敛，而 FP16 偶尔会溢出。

十、FlashAttention：让注意力计算变快

Attention 是 Transformer 的核心操作。标准实现需要计算一个 N×N 的注意力分数矩阵，把它完整地写到显存里再读回来——两次 $O(N^2)$ 的读写。当 N=4096 时没问题，当 N=128K 时这个矩阵有 16B 个元素，已经超过了大部分 GPU 的显存容量。

打个比方：**标准 Attention 做计算就像在冰箱和灶台之间反复跑——每次炒菜都要从冰箱拿一盘菜、切一半、放回去、再拿出来。FlashAttention 把菜一次性拿到灶台边的备菜台上，一次切完。** ——速度自然就快了。

FlashAttention 的解决方案：通过 tiling（分块），在 GPU 的高速缓存（SRAM）上完成 Attention 的子计算，只把最终结果写回显存。显存读写从 $O(N^2)$ 降到 O(N)。三个版本的演进：

- **FlashAttention 1**（2022, Dao et al.）：分块计算 + 不存完整 Attention 矩阵 → BERT 训练提速 15%
- **FlashAttention 2**（2023）：更好的并行策略 → 再提速 2×
- **FlashAttention 3**（2024）：利用 H100 FP8 Tensor Core 新硬件特性 → 再提速 1.5–2×

FlashAttention 揭示了训练优化的一个深层规律：**很多时候瓶颈不是算力不够，而是数据搬运太慢。** 优化显存访问模式往往比优化计算本身有更大的收益。这个洞察同样适用于训练的其他层面——通信优化、数据加载、梯度同步。

十一、主流训练框架

DDP vs FSDP vs DeepSpeed——有什么区别?

技术	做什么	谁做的	关系
DDP	数据并行的基本实现	PyTorch	每张卡存完整模型，梯度同步平均
FSDP	数据并行 + 参数分片	PyTorch（Meta）	等价于 ZeRO Stage 3，原生集成 PyTorch
DeepSpeed	全套优化方案	微软	ZeRO + Offload + MoE + RLHF 一站式

FSDP 是 PyTorch 对 ZeRO Stage 3 的重新实现。两者功能类似，但 FSDP 跟 PyTorch 生态捆绑更紧（更好兼容 torch.compile 等），DeepSpeed 则有更多高级功能（Offload、MoE、通信优化）。对于大多数中小团队，FSDP 是更简单的选择；对于 100B+ 模型或需要 Offload 的场景，DeepSpeed 是唯一选择。

完整框架对比

框架	开发方	核心优势	适用规模	并行策略	学习曲线
DeepSpeed（Microsoft, 2020）	微软	ZeRO 系列最完善，推理优化也做	1B–1000B	ZeRO/TP/PP	中等
Megatron-LM（Shoeybi et al., 2020）	NVIDIA	极致性能，TP 原创	10B–1000B	TP+PP+DP+EP	陡峭
FSDP	PyTorch 官方	最易用，零配置入门	1B–100B	FSDP(ZeRO-3)	低
JAX	Google	函数式，XLA 编译优化	1B–1000B	FSDP + 自定义	高

各框架详解

DeepSpeed 是目前最流行的训练库。它的 ZeRO 系列是行业标准——你把代码换成 `deepspeed.initialize()`，配置一个 ZeRO 级别，就能跑几百亿参数的模型。ZeRO-Offload 还能把优化器状态卸载到 CPU 内存中，让显存有限的 GPU 也能训大模型。但 DeepSpeed 的配置选项极多（`ds_config.json` 有几百个参数），调优需要经验。

Megatron-LM 是 NVIDIA 的旗舰训练框架。TP 和 3D 并行的首创者。每一行 CUDA kernel 都是手写优化的，性能是四个框架中最高的。代价是使用门槛极高——你得理解 TP/PP/DP 各自的角色，手动配置分片策略，改模型代码适配 TP。大厂 infra 团队的首选，中小团队慎入。

FSDP 是 PyTorch 官方方案，也是当前中小团队的主流选择。API 简洁，跟 PyTorch 生态无缝对接（Hugging Face Transformers 直接支持 FSDP）。性能比 DeepSpeed ZeRO-3 略低（约 5–10%，社区估算），但调试和维护成本低得多。如果你的模型在 70B 以下、节点规模在 32 以内，FSDP 大概率是最划算的选择。

JAX 走的是完全不同的路线。它不是"框架"，而是一套函数式编程工具链。你写纯函数描述计算过程，JAX 用 XLA 编译器自动优化执行。优点是非常灵活——可以自定义任意并行策略。缺点是你基本得自己实现所有训练基础设施——数据加载、模型 checkpoint、分布式通信。适合有 infra 团队的机构做研究实验。

十一、MoE 训练的特殊挑战

MoE（混合专家模型）的训练跟 Dense 模型不一样。它把 FFN 层替换成多个专家和一个路由器，每个 token 只激活部分专家。这让模型可以在不算大幅增加计算量的情况下扩大参数量。

但 MoE 的训练引入了几个特有的难题。

负载均衡（Load Balancing）

路由器决定每个 token 去哪个专家。问题在于：如果路由器学偏了，所有 token 都往同一个专家跑，其他专家闲着没事干——计算资源浪费了，模型能力也降低了。

解决方法：

- **Auxiliary Loss（辅助损失）**：在总 loss 中加入一项惩罚，鼓励路由器均匀分配。这是业界主流做法
- **Aux-Loss-Free（DeepSeek V3）**：在路由器中加一个 bias 项，专家被选得越多 bias 越大（降低它的选择概率），选得越少 bias 越小（提高概率）。不增加 loss 项，避免对主任务产生干扰

All-to-All 通信

在 MoE 的训练中，token 处理完自己的 expert 后可能要传给另一个 GPU 上的 expert。这叫做 All-to-All 通信——每个 GPU 都给每个其他 GPU 发数据。

```
/--- GPU 0 ---\          /--- GPU 3 ---\
| Expert A     | ←——→ | Expert D     |
| Token(猫)    |          | Token(狗)    |
\-------------/          \-------------/
     ↑ all-to-all              ↓
/--- GPU 1 ---\          /--- GPU 2 ---\
| Expert B     |          | Expert C     |
| Token(鱼)    |          | Token(鸟)    |
\-------------/          \-------------/
```

All-to-All 没有固定模式，不像 all-reduce（每张卡发一个值给所有人然后求和）。它的通信量取决于路由结果，工程师很难提前预估。DeepSeek V3 的 DualPipe 技术通过把 All-to-All 通信跟计算重叠来隐藏通信延迟。

Expert Capacity（专家容量）

每个专家能处理多少 token 是有限的（max capacity）。如果某个专家收到的 token 太多，超出的 token 有两种处理方式：

- **丢弃**：超出容量的 token 直接跳过这个 expert，通过 residual connection 传到下一层
- **重新路由**：试试能不能分给其他专家

容量设太大 = 显存浪费（每个专家都要预留足够空间）。容量设太小 = 太多 token 被丢弃，模型训练不充分。

训练不稳定

MoE 比 Dense 模型更难用。同样的学习率对 Dense 模型刚刚好，对 MoE 可能震荡。路由器在训练早期很不稳定——前几步的随机分配会影响后续 expert 的专业化方向。常用的技巧包括：

- 更小的学习率
- 更长的 warmup
- 更保守的初始化
- 路由器增加 noise（训练早期鼓励探索）

十二、一张图串起来

```
原始数据
  ↓
数据预处理 → Tokenize → 构建批次
  ↓
模型放置到 N 张 GPU
  ↓
并行策略选择（核心决策）
├── 单卡放得下 → 普通 DP 或 FSDP
├── 需要省显存 → ZeRO-1/2/3（DeepSpeed / FSDP）
├── 一层太大 → TP（切层内计算，需 NVLink）
├── 层数太多 → PP（分层放，跨节点友好）
├── 以上全要 → 3D 并行（TP + PP + DP）
└── MoE 模型 → 再加 EP（专家并行）
  ↓
训练循环（重复几十万次）

  ┌──────────────────────────────────────┐
  │  前向传播（Forward）                  │
  │    │                                 │
  │    ├─ 逐层计算                        │
  │    ├─ 混合精度：BF16 算，FP32 存      │
  │    ├─ 梯度检查点：不存中间激活值       │
  │    └─ MoE 时：路由 + All-to-All 通信   │
  │    ↓                                 │
  │  计算 Loss                            │
  │    ↓                                 │
  │  反向传播（Backward）                 │
  │    │                                 │
  │    ├─ 逐层算梯度                      │
  │    ├─ 梯度裁剪（max_norm=1.0）        │
  │    └─ FSDP/ZeRO 做 reduce-scatter     │
  │    ↓                                 │
  │  梯度同步（All-Reduce / Reduce-Scatter）│
  │    ↓                                 │
  │  优化器更新（AdamW）                  │
  │    ├─ LR 调度（Warmup + Cosine）      │
  │    └─ 更新参数 + 优化器状态           │
  │    ↓                                 │
  │  Loss 检查                            │
  │   ├─ NaN → 降低 LR / 回滚 checkpoint  │
  │   ├─ 不降 → 检查数据 / 增加 warmup    │
  │   └─ 正常 → 继续下一轮                │
  └──────────────────────────────────────┘

    ↓
保存 Checkpoint（每 N 步）
  ├─ 模型权重（BF16）
  ├─ 优化器状态（FP32）
  ├─ 训练进度（step / LR / loss）
  └─ 恢复时可从任意 checkpoint 继续
    ↓
训练完成 → 模型评估 → 推理部署
```

从数据到训完的模型，每一步都在回答一个问题：**如何在显存、算力、通信的铁三角中扛住几千亿参数和上万亿 token 的考验，让运行几周的训练不崩、不慢、不浪费 GPU。**

与推理前置知识的关联

这篇文档和 大模型推理部署前置知识 是**互补关系**——前者讲"怎么把模型训出来"，后者讲"怎么把训好的模型上线"。

建议阅读顺序

1. 先读 Transformer 前置知识，理解模型内部原理
2. 再读本文，理解模型怎么训练
3. 再读 大模型推理部署前置知识，理解模型怎么上线
4. 对照 MoE 前置知识 理解混合专家模型的特殊性

> **接下来**：大模型训练主报告 — 深入训练框架选型、并行策略决策、面试实战。或 大模型推理部署前置知识 — 如何在显存、算力、延迟的铁三角中找出推理最优解。

参考来源

- **Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism** (Shoeybi et al., 2020) — 首创张量并行 + 流水线并行的 3D 并行训练框架 — https://arxiv.org/abs/1909.08053
- **ZeRO: Memory Optimizations Toward Training Trillion Parameter Models** (Rajbhandari et al., 2020) — 数据并行的显存冗余消除，ZeRO-1/2/3 三级优化 — https://arxiv.org/abs/1910.02054
- **GPipe: Efficient Training of Large Neural Networks using Pipeline Parallelism** (Huang et al., 2019) — 流水线并行的奠基工作，微批次调度与气泡分析 — https://arxiv.org/abs/1811.06965
- **Mixed Precision Training** (Micikevicius et al., 2018) — FP16 混合精度训练，前向反向用 FP16、权重更新用 FP32 的标准范式 — https://arxiv.org/abs/1710.03740
- **Training Compute-Optimal Large Language Models** (Hoffmann et al., 2022) — Chinchilla 缩放定律，给定算力预算下模型参数与训练 token 的最优配比 — https://arxiv.org/abs/2203.15556
- **DeepSpeed** (Microsoft, 2020) — ZeRO 系列 + 训练推理优化一体化框架 — https://www.deepspeed.ai/
- **NVIDIA H100 Tensor Core GPU Architecture Whitepaper** (NVIDIA, 2022) — Hopper 架构官方白皮书，H100 FP16 算力约 1979 TFLOPS，NVLink 4 带宽 900 GB/s — https://resources.nvidia.com/en-us-tensor-core/gtc22-whitepaper-hopper

- **NVIDIA Blackwell Architecture Technical Overview** (NVIDIA, 2024) — Blackwell/B200 双 die 封装，FP4 算力约 4500 TFLOPS，NVLink 5 带宽 1.8 TB/s — https://resources.nvidia.com/en-us-blackwell-architecture
- **FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness** (Dao et al., 2022) — 通过分块计算将 attention 显存读写从 $O(N^2)$ 降至 $O(N)$ — https://arxiv.org/abs/2205.14135
- **FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning** (Dao, 2023) — 改进并行策略实现 2× 加速 — https://arxiv.org/abs/2307.08691
- **FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-Precision** (Dao et al., 2024) — 利用 H100 FP8 Tensor Core 实现 1.5-2× 再加速 — https://arxiv.org/abs/2407.08608